Relational Database Systems –

COSC265

Lectures 25 & 26 – Indexing Structures



Miguel Morales

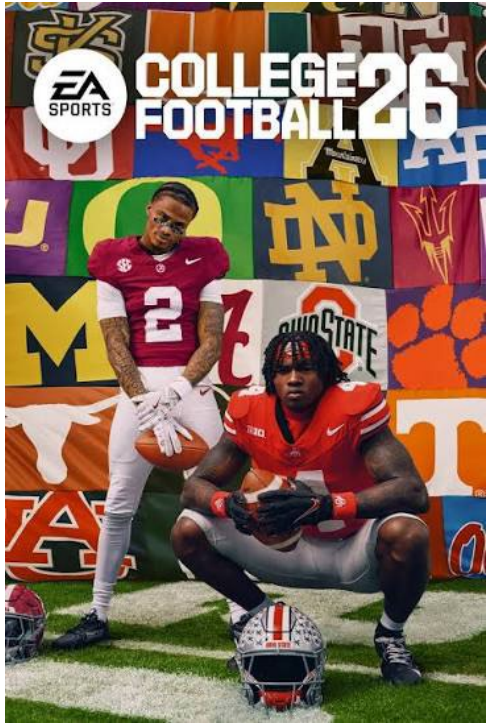
UC UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

[Un]Ordered Files

- Imagine **10,000,000** College Football player records physically ordered by Player ID

- Find player 3867321
  - **Nice**, the physical order helped a lot. It took us less than a second

- Find all players who study/studied at UCLA
  - **Painful**; the order is useless in this case. It took us more than 10 seconds
  - This type of query (i.e., players from a particular university) is one of the most used during the recruitment period, peaking at 300 requests per minute



Should we reorder the entire file by University?

Any ideas on how to solve this?

Index Structures

- The index is an **ordered file with two fields**
  - A **key** value
  - A **pointer** to a data block

- **Indexes** are used to speed up record retrieval in response to certain search conditions
  - Can be compared to the index of a book
  - **Any field can be used** to create an index; the chosen one will be called the **indexing field**
  - Multiple indexes can be constructed per table
  - **An index is a new structure**; i.e., it is physically stored
  - Since **index entries are ordered** by the indexing field value, **binary search can be used**

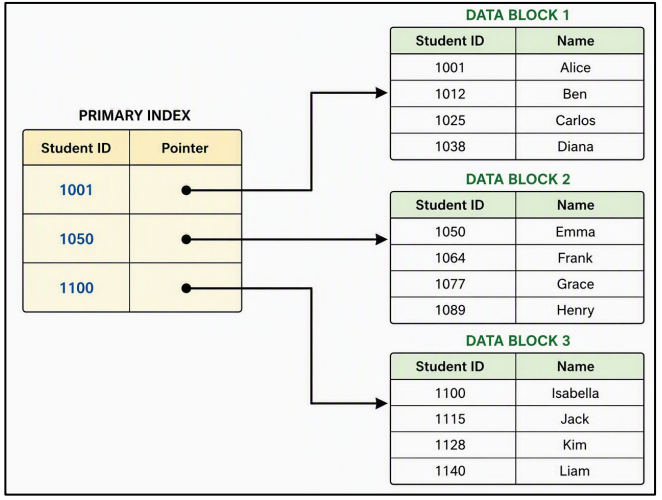
Types of Indexes

- **Primary** index
  - Specified **on the ordering field** of an ordered file
- **Clustering** index
  - Used if **multiple records** can have the **same value** for the ordering field (i.e., the field is not a CK)
- **Secondary** index
  - Can be specified **on any non-ordering field**
  - A data file can have several secondary indexes

Primary Indexes

- A **primary index** is composed of:
  - A **key**, which is a CK and the ordering field of the data file
  - A **pointer** to a data block

- Indexes can be:
  - A **dense** index has **an index entry for every search key value** in the data file
  - A **sparse** (or non-dense) index has **entries for only some key values**

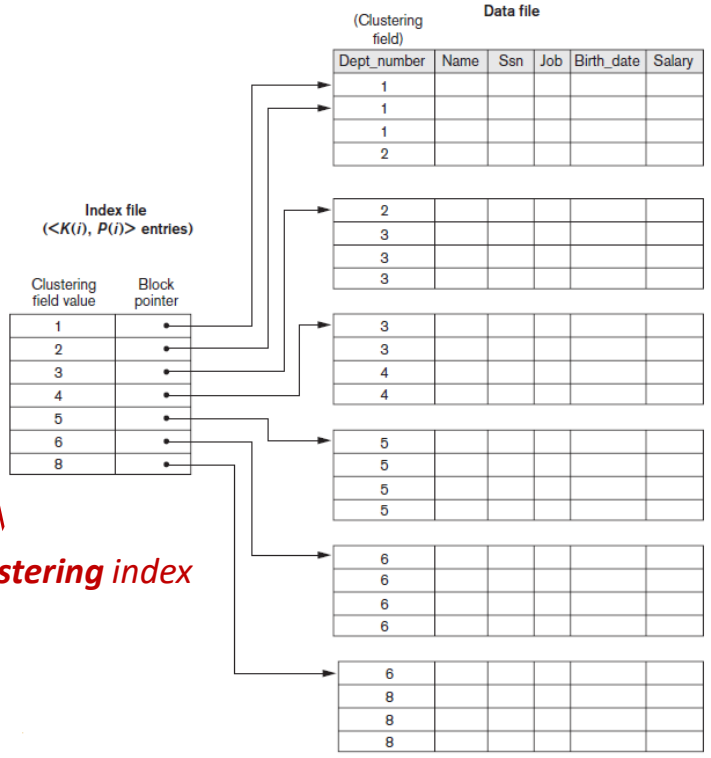


*This is a **primary** and **sparse** index →*

How efficient are insertion, deletion, and search?

Clustering Indexes

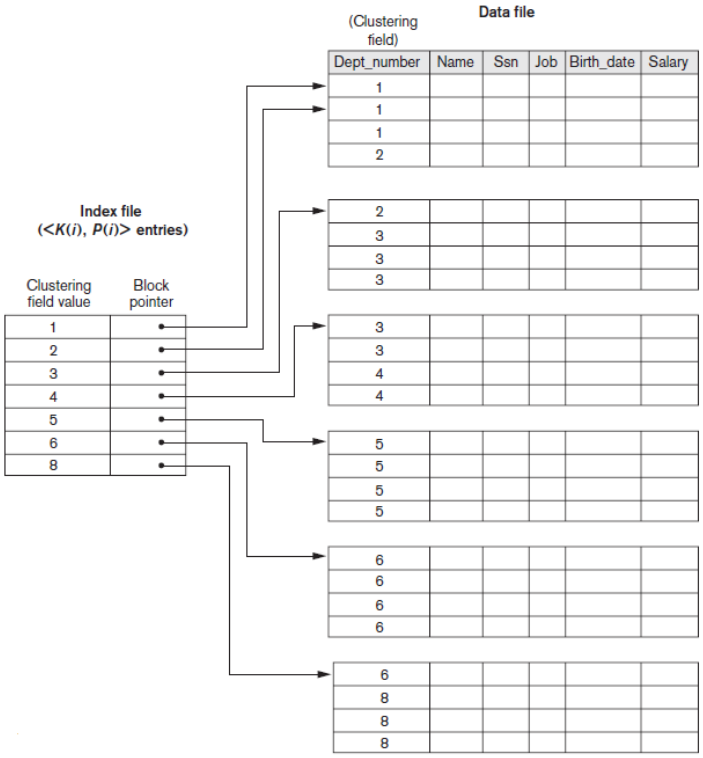
- Here, the data file records are:
  - Physically **ordered on a non-key field** (the *clustering field*)
  - Since it is a non-key field, the same value for that field may appear multiple times
- A **clustering index** is then formed by:
  - A **key**, which is the clustering field
  - A **pointer** to a data block



*This is a **clustering** index*

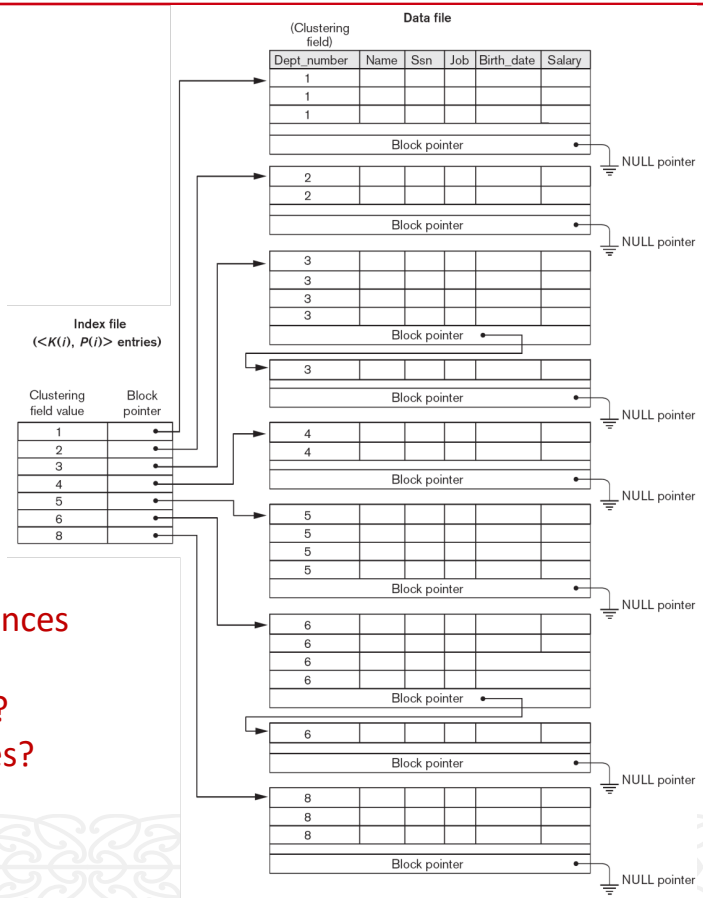
How efficient are insertion, deletion, and search?

Clustering Indexes



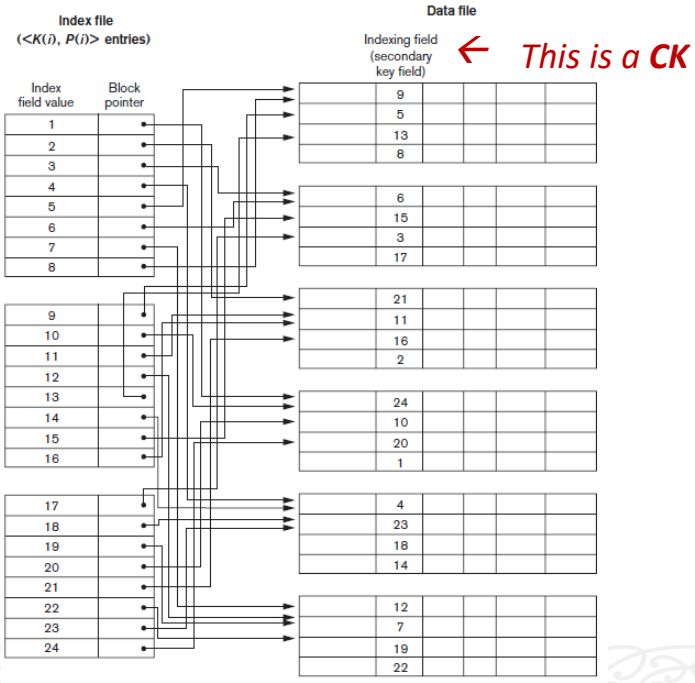
Spot the differences

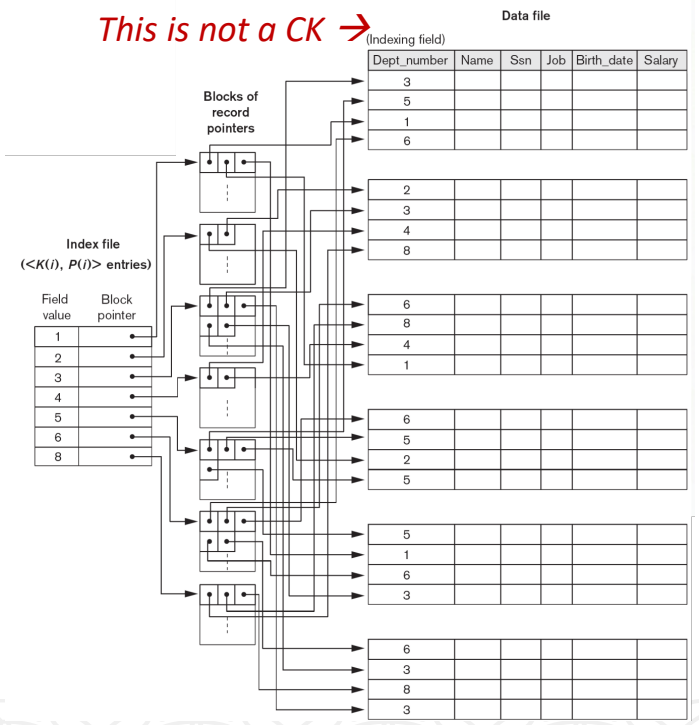
Advantages?
Disadvantages?



Secondary Indexes

- A primary index exists, so secondary indexes provide an alternative for accessing the data file





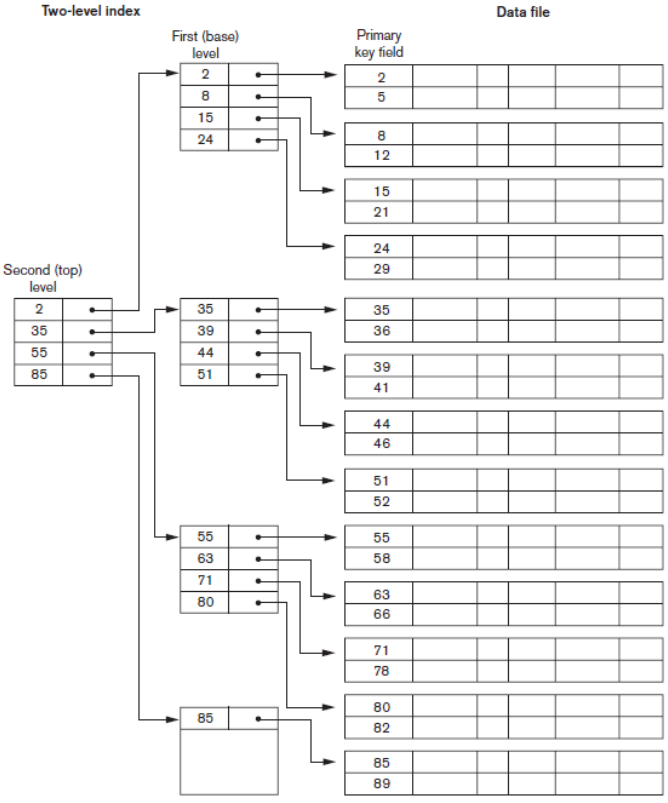
Morale Homework 1

UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

- Compare the speed of retrieval for a file with 30,000 records of fixed length 100B, given a block size of 1024B, an index size of 9B, and a pointer size of 6B.
- In the case of:
  - an ordered file
  - a file with a primary index
  - a file with a secondary index
- Assume unspanned blocks.

Multi-level Indexes

- Because a single-level index is an ordered file, we can create **an index to the index**
  - The original index file is called the first-level index
  - The *index to the index* is called the second-level index
  - Repeat the process **until all entries fit in one disk block**
- These can be created for any type of first-level index
  - Search tree
  - Fan-out
- Insertion and deletion become a problem because every level of the index is an ordered file

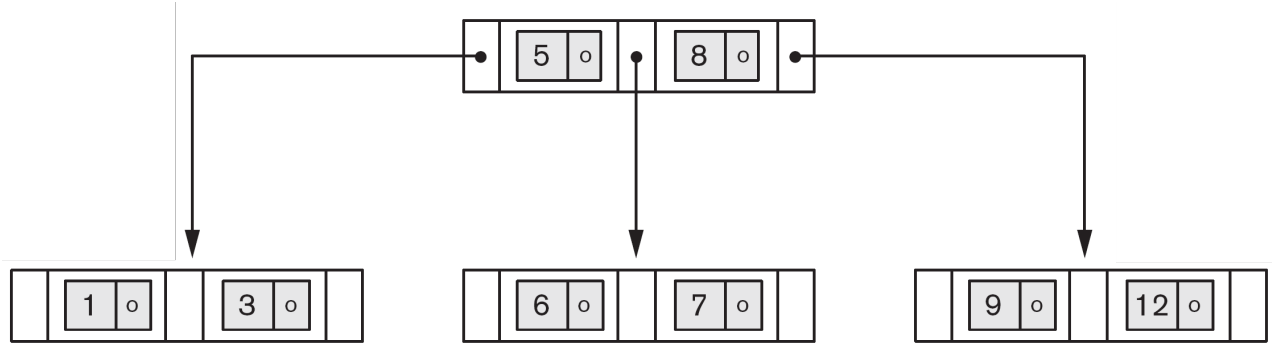


Morale Homework 2

- How many levels of indexing would be needed for the file from Morale Homework 1?

B -Trees and B+ -Trees

- Most multi-level indexes use B-tree or B+-tree data structures because of the insertion and deletion costs
  - In **B-Tree** and **B+-Tree** data structures, **each node corresponds to a disk block**
  - Generalizations of binary search trees
  - Each node is kept **between 50% and 100% full**

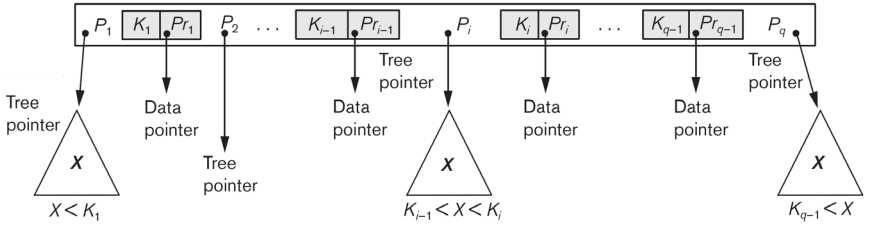


B -Trees

- **Node structure** in a B-tree of order q:

$$<P_1, <K_1, Pr_1>, P_2, <K_2, Pr_2>, P_3, \ldots, <K_{q-1}, Pr_{q-1}>, P_q>$$

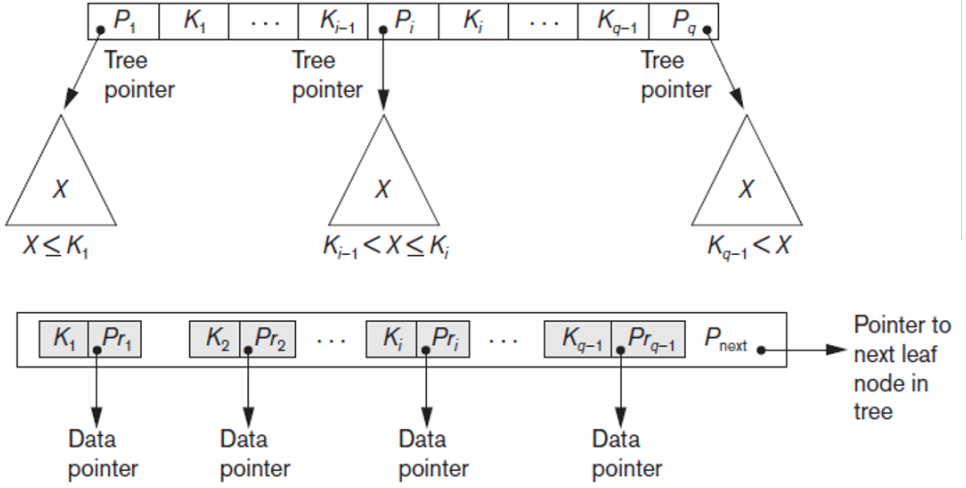
- Where:
  - P_i is a tree pointer
  - K_i is a key value
  - Pr_i is a data pointer



- Keys are ordered: $K_1 < K_2 < \ldots < K_{q-1}$
- Each node in a B-tree of order q can have at most q-1 key values
- The tree is **balanced** (i.e., all leaf nodes are at the same level)
- **Data pointers are present at any level**

B+ -Trees

- Data pointers are stored **only at leaf nodes**
- Leaf nodes have:
  - K_i a search value
  - Pr_i a data pointer
- For a **non-CK search field**, the pointer Pr_i points to a block containing pointers to the data file records
- Search, insertion, and deletion become [very] efficient



Morale Homework 3

UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

- Calculate the order of a B+ -tree given:
  - Block size of 512
  - Index size of 9B
  - Pointer size of 7B
  - Index tree pointer size of 6B

- Approximately, in this B+ -tree, how many levels would we need to store 1 million records?

SQL Index Creation

- By default, when a table is created, an index is generated on the PK of the table

```
-- Creating an index on the player's last name attribute
CREATE INDEX my_index ON player(player_lastname);

-- Removing the index
DROP INDEX my_index;
```

